

Apex Classes and Apex Triggers

Apex Classes

- Search covers name and status. Selecting a class retrieves Body and shows it in a scrollable, copyable source panel.
- The summary identifies visible methods and inputs. Expandable sections cover execution markers, SOQL/read objects, DML/write markers, callouts and async work, explicit error handling, and dependencies.
- References include method names and one-based source line numbers for identifiable declarations and operations.
- Dependencies are inferred from class-qualified calls. Platform built-ins are filtered, but dynamic dispatch and reflection can hide relationships.

Apex Triggers

- The trigger declaration parser identifies the sObject and declared before/after events and summarizes when the trigger runs.
- SOQL, DML, callout/async, and error-handling sections reuse the Apex structural analyzer.
- Class-qualified references are presented as possible handler classes. Each handler is a link to the Apex Classes subtab using explorerName; retrieval still succeeds only when the user can access that class.
- If a declaration or behavior cannot be confidently recognized, the UI displays an Uncertain label and preserves the original source for verification.

Interpretation boundaries

Reported directly	Inferred or potentially incomplete
Names, status, API version metadata, trigger object/events, source text, explicit SOQL/DML/callout/try-catch markers.	Business purpose, effects hidden behind called code, dynamic SOQL, reflection, dependency injection, managed-package internals, runtime branch outcomes.
Method names and cited source lines.	A class-qualified reference is a likely dependency; it is not proof that the path executes in every transaction.

Error and permission behavior

- ConnectionGate blocks retrieval without a selected connected org. Salesforce authorization controls list and body visibility.
- Typed authentication, permission, network, cancellation, limit, and server errors use the shared ErrorAlert behavior. Refresh retries the selected definition.
- An inaccessible handler remains a navigation attempt that resolves to no accessible class rather than exposing hidden source.